



Documentacion Tecnica

Red Sanitaria Bolivia

Sistema de Optimizacion Logistica

Central de Abastecimiento y Suministros en Salud

INFORMACION DEL PROYECTO

Universidad: UPEA - Ingenieria de Sistemas

Docente: M.Sc. Ing. Juan Carlos Catunta Choquecalle

Responsable: Elias Quispe Quispe

Fecha: 24/9/2026

Version: 5.0

- 1. Introduccion y Objetivos**
- 2. Arquitectura General**
- 3. Estructuras de Datos**
- 4. Ubicacion delCodigo Fuente**
- 5. Los 5 Algoritmos**
 - 5.1 BFS - Amplitud
 - 5.2 DFS - Profundidad
 - 5.3 Dijkstra - Ruta Minima
 - 5.4 Kruskal - MST Global
 - 5.5 Prim - MST Local
- 6. Flujo de Ejecucion**
- 7. Modulos Funcionales**
- 8. Atajos de Teclado**
- 9. Exportaciones**
- 10. Glosario**

1. Introduccion y Objetivos

¿Que es este sistema?

La aplicacion "Red Sanitaria Bolivia - CEASS" modela la red de distribucion de medicamentos de la Central de Abastecimiento y Suministros en Salud de Bolivia como un grafo ponderado. Sobre este grafo se aplican 5 algoritmos clasicos de teoria de grafos para optimizar rutas y diseñar la red de distribucion de la manera mas eficiente posible.

Objetivo general

Proporcionar una herramienta interactiva que permita visualizar, analizar y optimizar la red de abastecimiento de medicamentos del CEASS, aplicando algoritmos de busqueda, caminos minimos y arboles de expansion minima.

Objetivos especificos

- Modelar la red de hospitales, hubs, SEDES y fronteras como un grafo ponderado.
- Implementar 5 algoritmos clasicos de teoria de grafos.
- Permitir edicion manual de la red (agregar/eliminar nodos y aristas).
- Exportar resultados en PNG, PDF e informes de texto.

Tecnologias utilizadas

HTML5 + CSS3 + JavaScript (vanilla). Librerias: Leaflet 1.9.4 (mapas interactivos), Chart.js (graficos), jsPDF 2.5.1 (generacion de PDF), html2canvas (capturas de pantalla), Font Awesome 6.5.1 (iconos), Google Fonts - Inter (tipografia).

Estructura del proyecto

Todo el sistema esta contenido en un UNICO archivo HTML (index.html) que incluye el CSS, el HTML y el JavaScript. Esto facilita la portabilidad: solo se necesita abrir el archivo en un navegador moderno (Chrome, Firefox, Edge).

Organizacion del archivo index.html

- Bloque <head>: metadatos, CDN de librerias, hoja de estilos <style>.
- Bloque <body>: estructura visual (splash, dashboard, paneles, modales).
- Bloque <script>: toda la logica JavaScript (variables, funciones, algoritmos).

Flujo general

Usuario abre index.html !' Splash screen (2.4s) !' Carga automatica de datos d
configura criterio y algoritmo !' Click "Ejecutar" !' Resultado en mapa y panele

3. Estructuras de Datos

3.1 redSalud (nodos del grafo)

Objeto que contiene todos los nodos de la red. Cada nodo tiene: nombre, tipo, latitud y longitud.

```
redSalud = {  
  'LPZ': { nombre:'La Paz - CEASS', tipo:'Hub CEASS',  
           lat:-16.5000, lng:-68.1500 },  
  ...  
}
```

3.2 adyacencia (aristas del grafo)

Matriz dispersa que guarda las conexiones entre nodos. Si $\text{adyacencia}[A][B] = 0.5$, significa que A conecta con B con peso 0.5 (horas por defecto).

```
adyacencia = {  
  'LPZ': { 'EAL':0.5, 'HCL':0.3, 'HNORTE':0.4 },  
  'EAL': { 'LPZ':0.5, 'ORU':3.0 },  
  ...  
}
```

3.3 pesosMultiples

Para cada arista se calculan 4 pesos distintos segun el criterio seleccionado: tiempo (h), distancia (km), costo (Bs) y riesgo (1-10).

```
pesosMultiples['LPZ-EAL'] = {  
  tiempo: 0.5, distancia: 30,  
  costo: 44, riesgo: 2  
}
```

4. Ubicacion delCodigo Fuente

El archivo index.html esta dividido en secciones. La siguiente tabla indica en que linea aproximada esta cada parte:

SECCION	LINEA
CSS (estilos)	14 - 370
HTML Body	375 - 1120
Variables globales	1126 - 1145
Animacion de fondo	1147 - 1195
Utilidades (toast, log)	1197 - 1280
Historial y actividad	1227 - 1300
Ponderacion	1302 - 1360
Modales	1362 - 1400
Mapa (Leaflet)	1402 - 1460
Actualizacion del grafo	1465 - 1580
Carga de datos (CSV)	1585 - 1780
>> ALGORITMOS <<	1812 - 2075
ejecutar()	1812 - 1830
runBFS()	1832 - 1880
runDFS()	1882 - 1930
runDijkstra()	1932 - 1980
runKruskal()	1982 - 2030
runPrim()	2032 - 2075
Analisis estadisticos	2077 - 2160
Comparacion	2162 - 2360
Graficos (Chart.js)	2362 - 2390
Exportaciones	2392 - 2540
Editor CRUD	2562 - 2800
Modo mapa	2862 - 2920
Inicializacion	2922 - 2970

5. Los 5 Algoritmos

La seccion mas importante del sistema. Los algoritmos se ejecutan llamando a la funcion ejecutar(), que decide cual correr segun el radio button seleccionado.

5.1 BFS - Busqueda en Amplitud

Funcion: runBFS(start) | Linea: 1832 | Color: #0066FF (azul)

Recorre el grafo nivel por nivel desde el nodo origen. Todos los vecinos directos son nivel 1, sus vecinos nivel 2, y asi sucesivamente. Util para visualizar como se propaga el abastecimiento desde un hub.

Estructura: Cola FIFO | Complejidad: $O(V + E)$ | Uso: Explorar red por capas.

5.2 DFS - Busqueda en Profundidad

Funcion: runDFS(start) | Linea: 1882 | Color: #00CC44 (verde)

Se interna lo mas profundo posible por una rama. Cuando no puede continuar, retrocede y explora otra rama. Implementado con recursion.

Estructura: Recursion (pila) | Complejidad: $O(V + E)$ | Uso: Encontrar caminos alternativos.

5.3 Dijkstra - Ruta Minima

Funcion: runDijkstra(start, target) | Linea: 1932 | Color: #FF6600 (naranja)

Encuentra el camino mas corto (menor peso) entre dos nodos. Usa una cola de prioridad implicita: siempre expande el nodo no visitado con menor distancia acumulada.

Estructura: Cola de prioridad | Complejidad: $O(V^2)$ | Uso: LA RUTA OPTIMA.

ADVERTENCIA: No funciona con pesos negativos.

5.4 Kruskal - MST Global

Funcion: runKruskal() | Linea: 1982 | Color: #8800FF (morado)

Construye el arbol de expansion minimo (MST) considerando TODAS las aristas de la red globalmente. Ordena las aristas por peso ascendente y las agrega si no forman ciclo.

Estructura: Union-Find | Complejidad: $O(E \log E)$ | Uso: Diseñar red completa optima.

5.5 Prim - MST Local

Funcion: runPrim(start) | Linea: 2032 | Color: #FF0033 (rojo)

Igual que Kruskal en objetivo (MST) pero el arbol crece desde un nodo inicial especifico. Siempre agrega la arista mas barata que conecta el arbol actual con un nodo nuevo.

Estructura: Voraz desde un punto | Complejidad: $O(V^2)$ | Uso: Expansion desde un hub.

Tabla comparativa de algoritmos

ALGORITMO	ESTRUCTURA	COMPLEJIDAD	BUSCA
BFS	Cola FIFO	$O(V+E)$	Recorrer
DFS	Recursion	$O(V+E)$	Recorrer
Dijkstra	Cola prioridad	$O(V^2)$	Ruta minima
Kruskal	Union-Find	$O(E \log E)$	MST global
Prim	Voraz local	$O(V^2)$	MST local

Secuencia tipica de uso

- Paso 1: El usuario abre index.html en el navegador.
- Paso 2: Se muestra el splash screen durante 2.4 segundos.
- Paso 3: Se inicializan el mapa, los 5 mini-mapas, el grafico y los criterios.
- Paso 4: Se cargan automaticamente 40 nodos de prueba.
- Paso 5: El usuario selecciona criterio (tiempo/distancia/costo/riesgo).
- Paso 6: El usuario elige el algoritmo deseado (BFS/DFS/Dijkstra/Kruskal/Prim).
- Paso 7: El usuario selecciona origen y destino (destino solo para Dijkstra).
- Paso 8: Click en "Ejecutar" o atajo Ctrl+E.
- Paso 9: El algoritmo se ejecuta y dibuja su resultado en el mapa principal y en el mini-mapa correspondiente.
- Paso 10: El panel "Resultados" muestra el detalle numerico.
- Paso 11: El grafico comparativo se actualiza con las barras de colores.
- Paso 12: La accion queda registrada en el historial de actividad.

Llamada interna de ejecutar()

ejecutar() lee el radio button seleccionado, valida origen y destino, limpia polilineas anteriores del mapa, y llama a runBFS(), runDFS(), runDijkstra(), runKruskal() o runPrim() segun corresponda.

7. Modulos Funcionales y Atajos

Modulos del sistema

MODULO	FUNCION
Grafo	Modela red (nodos + aristas)
Mapa Leaflet	Visualizacion geografica
Algoritmos	5 algoritmos de grafos
Analisis	Estadisticas automaticas
Comparacion	Ejecuta los 5 en paralelo
Editor CRUD	Agregar/eliminar manualmente
Modo mapa	Click en mapa = lat/lng auto
Exportacion	PNG, PDF e informes TXT

Atajos de teclado

ATAJO	ACCION
Ctrl + E	Ejecutar algoritmo seleccionado
Ctrl + R	Reiniciar vista
Ctrl + D	Cargar datos de prueba
Ctrl + M	Pantalla completa del mapa
Ctrl + K	Abrir editor de grafo
F1	Ver atajos
Esc	Cerrar modales / salir pantalla completa

Tipos de exportacion

- PNG: captura del mapa principal usando html2canvas.
- PDF: reporte de 5 paginas con portada, resumen, comparacion, ruta y recomendaciones.
- TXT: informe basico con conteo de nodos y aristas.
- CSV: dos archivos (vertices.csv y aristas.csv) descargables como plantilla.

Glosario

Nodo / Vertice:	Punto de la red (hub, hospital, frontera)
Arista:	Conexion entre 2 nodos con un peso
Grafo ponderado:	Grafo donde cada arista tiene un costo
MST:	Minimum Spanning Tree (arbol minimo)
Relajacion:	Actualizar distancia si hay mejor ruta
Union-Find:	Estructura para detectar ciclos
Hub:	Centro de distribucion principal
SEDES:	Servicio Departamental de Salud
Frontera:	Paso aduanero / aduana
Criterio:	tiempo/distancia/costo/riesgo

Fin del documento. Generado automaticamente por el sistema CEASS el 24/9/2026, 4:16:57 a. m..